Microprocessor & Computer Architecture (µpCA)

UE24CS251B

Session - 8

Microprocessor & Computer Architecture (µpCA)

UE24CS251B

Procedure Call or Subroutine Call or Function Call

Procedure Call or Subroutine Call or Function Call

- The single variant on the branch theme is the option to perform a link operation before the branch is executed. (BL)

- This simply means storing the current value of **R15 (PC) in R14** before the branch is taken, so that the program can resume its operation.

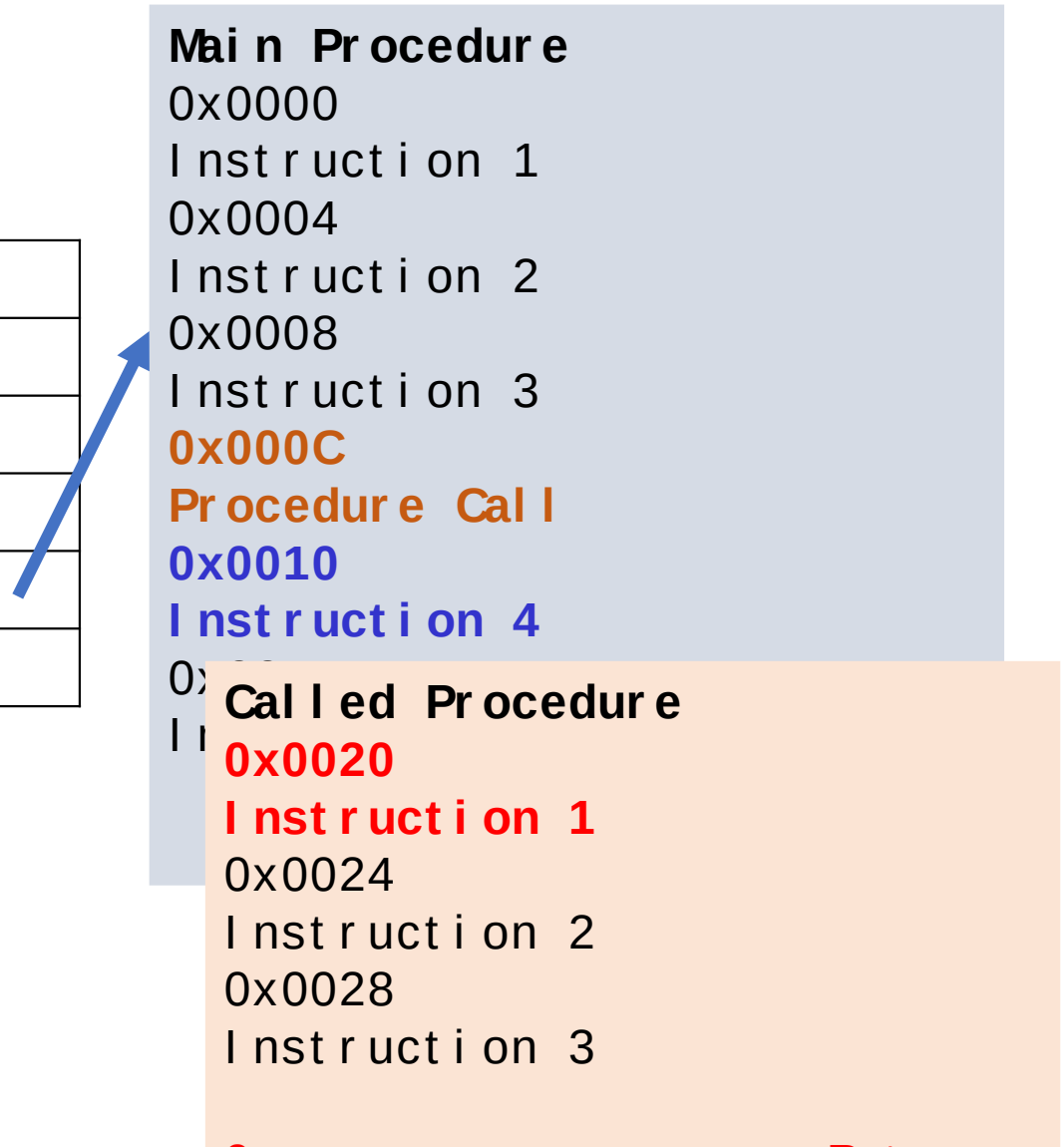
  - Procedure call is invoked using a BL instruction.
    - Branch with link, **BL** instruction i.e.,
    - **BL subroutine**

Procedure Call or Subroutine Call or Function Call

- ARM has no dedicated stack.
  - It copies R15 into R14
- If the called Routine needs to use R14 for something, it can save it on the stack explicitly.
- The ARM method has the advantage that subroutines which don't need to save R14 can be called return very quickly.
- The disadvantage is that all other routines have the overhead of explicitly saving R14.
- To return from a subroutine, the program simply has to move R14 back into register R15.
  - MOV R15, R14 or MOV PC, LR
  - MOVS R15, R14 or MOV PC, LR

PROCEDURE CALL

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	**0x0010**
CPSR	



General Structure of Procedure Call :

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	**0x0010**
R15 (PC)	**0x0020**
CPSR	

Main Procedure
0x0000
Instruction 1
0x0004
Instruction 2
0x0008
Instruction 3
0x000C
Procedure Call
0x0010
Instruction 4

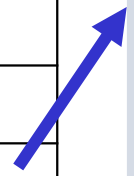
Called Procedure
0x0020
Instruction 1
0x0024
Instruction 2
0x0028
Instruction 3

LR=PC

PC= Address of the 1st Instruction

General Structure of Procedure return:

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	**0x0010**
CPSR	



Main Procedure
0x0000
Instruction 1
0x0004
Instruction 2
0x0008
Instruction 3
0x000C
Procedure Call
0x0010
Instruction 4
0x
Ins

Called Procedure
0x0020
Instruction 1
0x0024
Instruction 2
0x0028
Instruction 3

PC=LR

General Structure of Procedure call and return:

Main Procedure

0x0000
Instruction 1
0x0004
Instruction 2
0x0008
Instruction 3
0x000C
BL Procedure
0x0010
Instruction 4
0x0014
Instruction 5
:::::::::::::::::::
Instruction Last

Called Procedure

0x0020 **Procedure:**
Instruction 1
0x0024
Instruction 2
0x0028
Instruction 3
0xxxxx
MOV PC LR

or

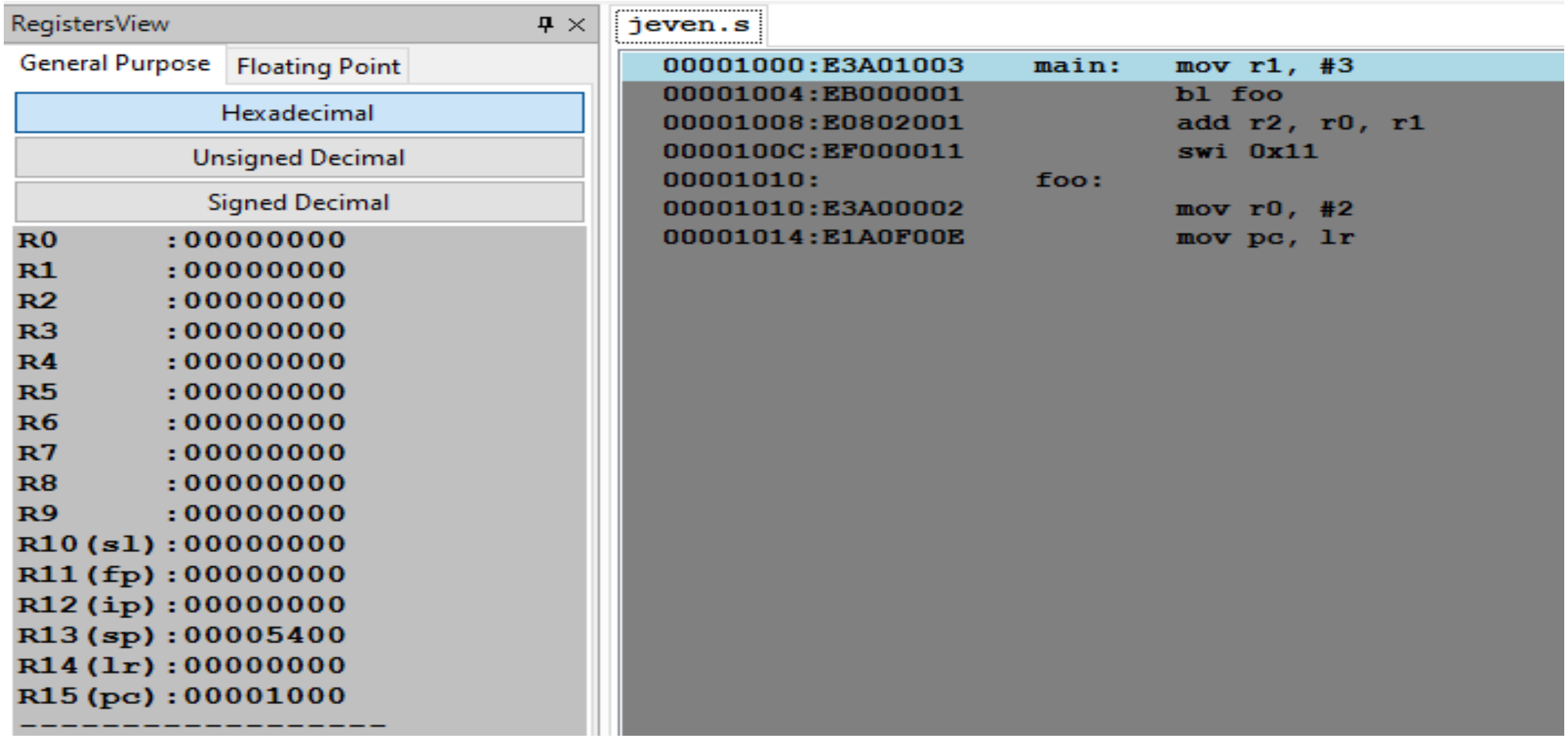
BX LR

PROCEDURE CALL EXAMPLE: 01

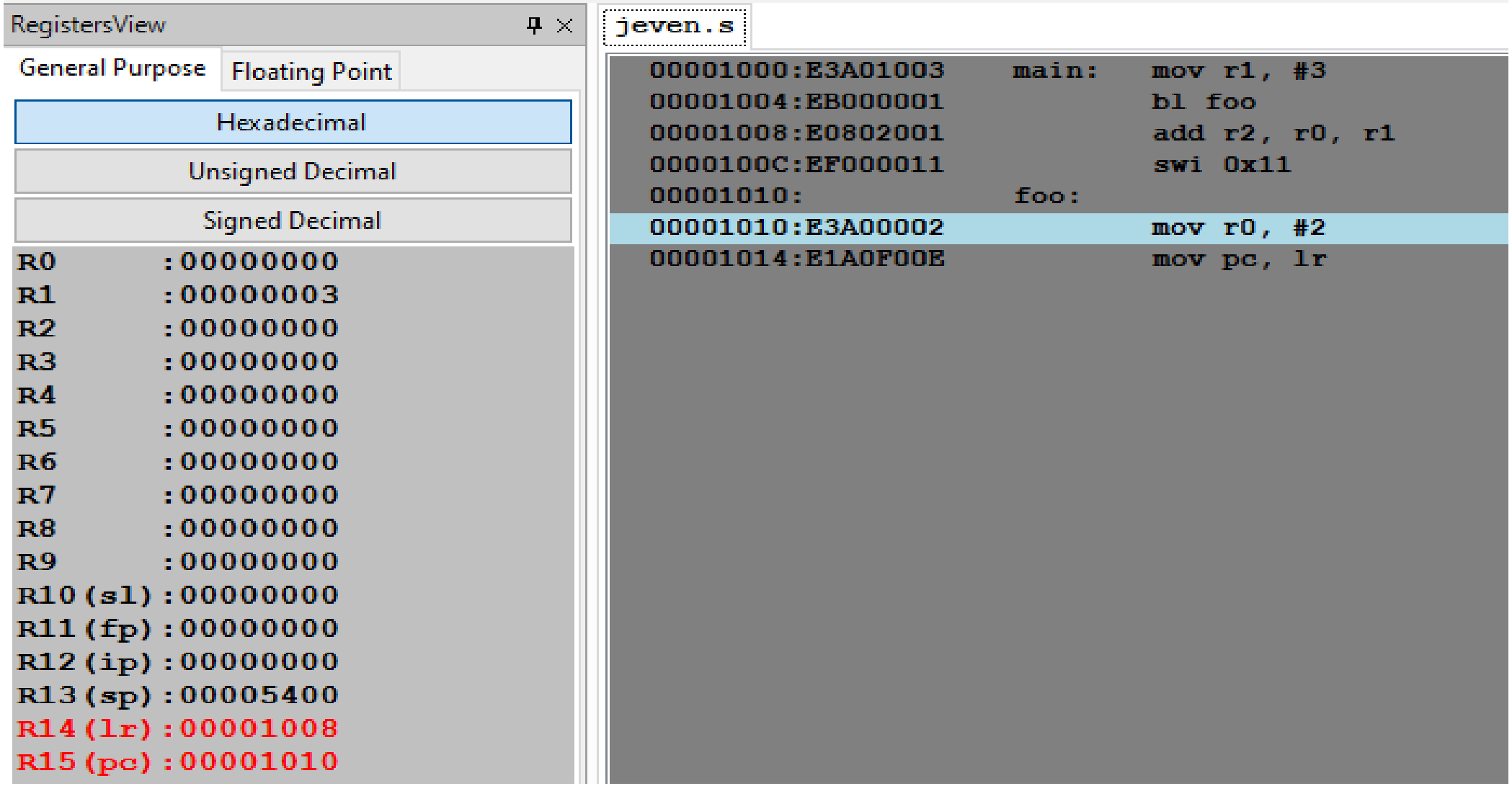
```
        MOV   R1, #3
        BL    FOOO
        ADD   R2, R0, R1
              SWI   0x11
FOOO:
        MOV R0, #2
        BX LR
```

```
MOV   R1, #3
      BL    FOOO
      ADD   R2, R0, R1
            SWI   0x11
FOOO:
      MOV R0, #2
      MOV PC, LR
```

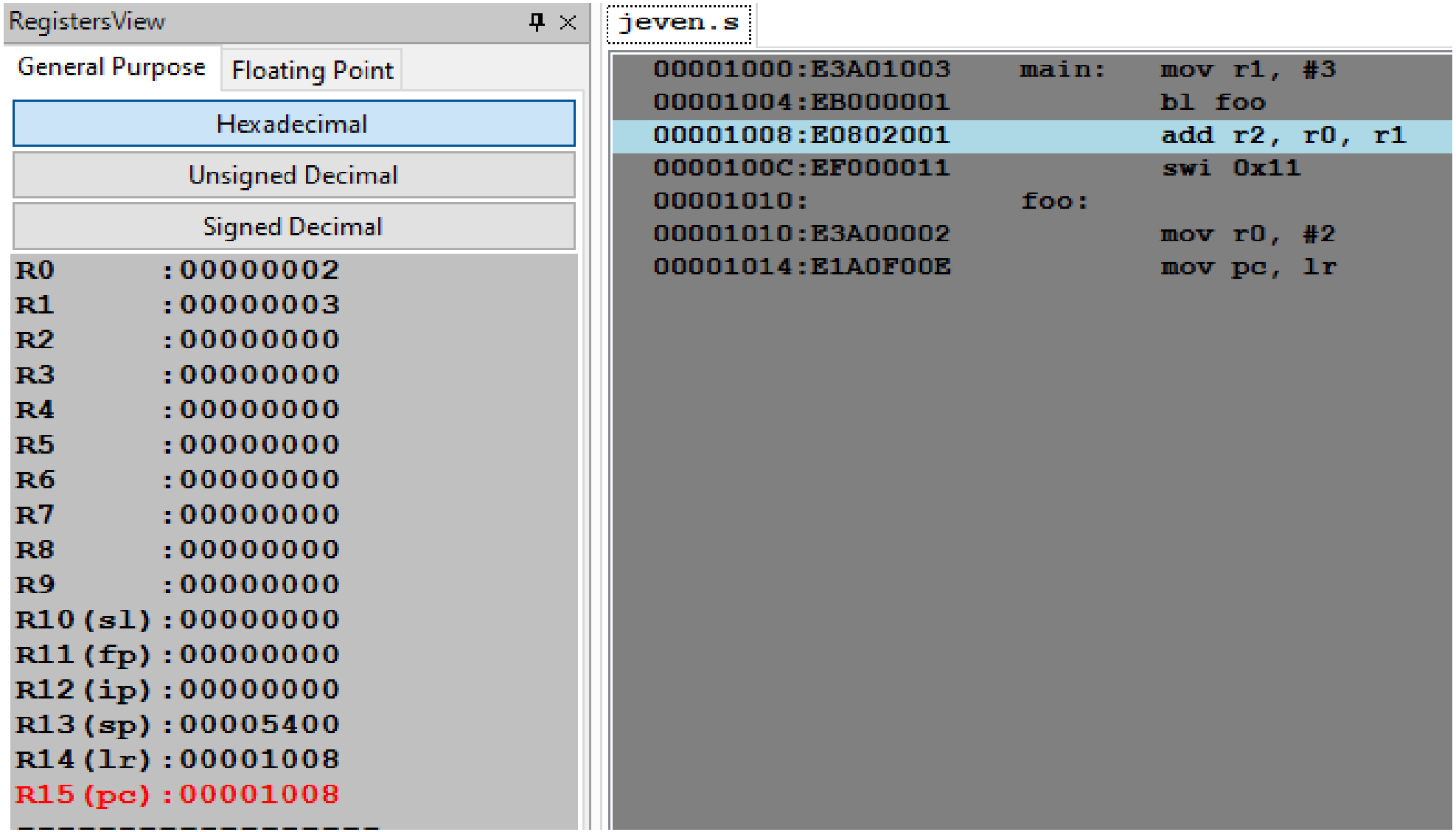
PROCEDURE CALL EXAMPLE: 01



PROCEDURE CALL EXAMPLE: 01



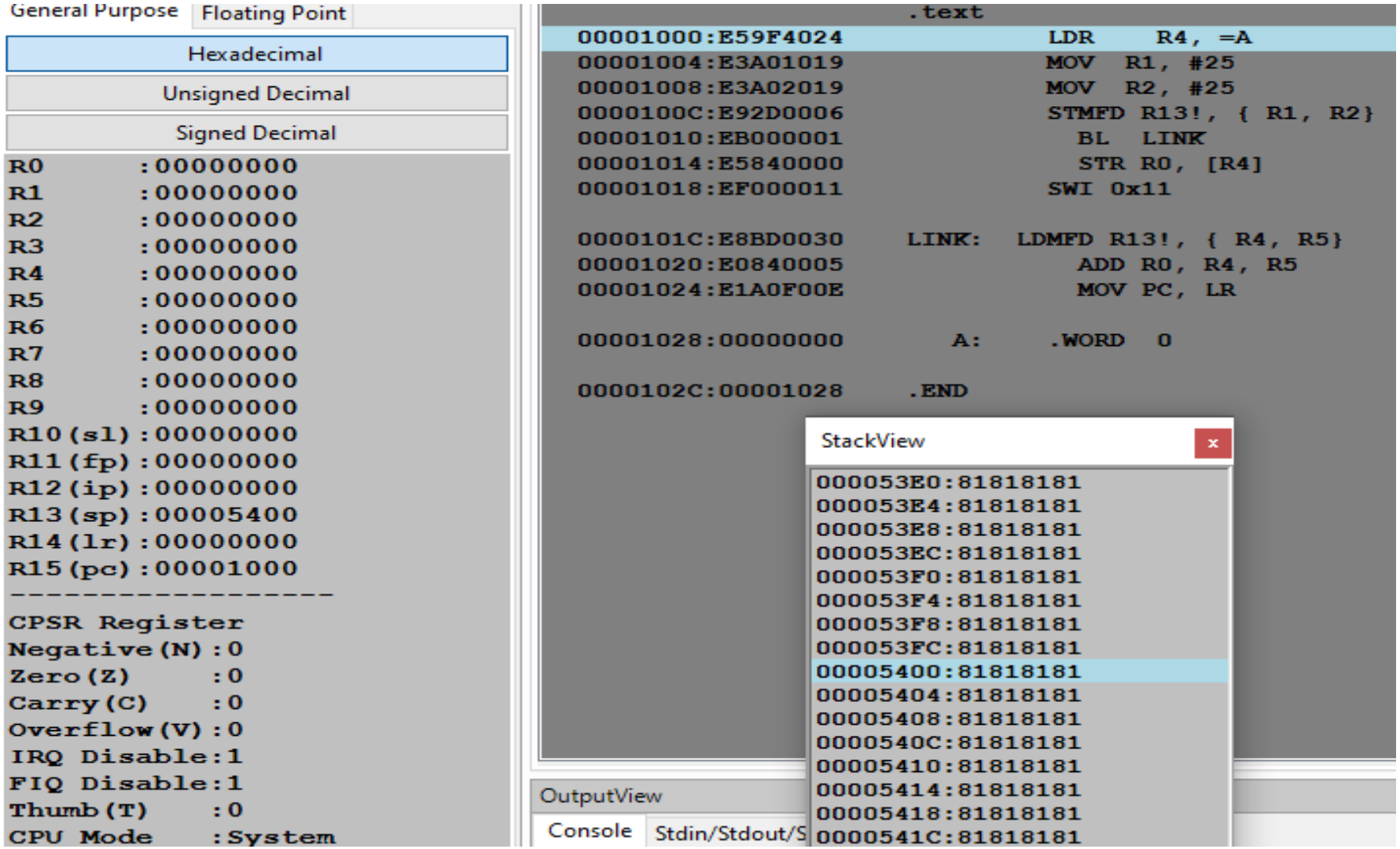
PROCEDURE CALL EXAMPLE: 01



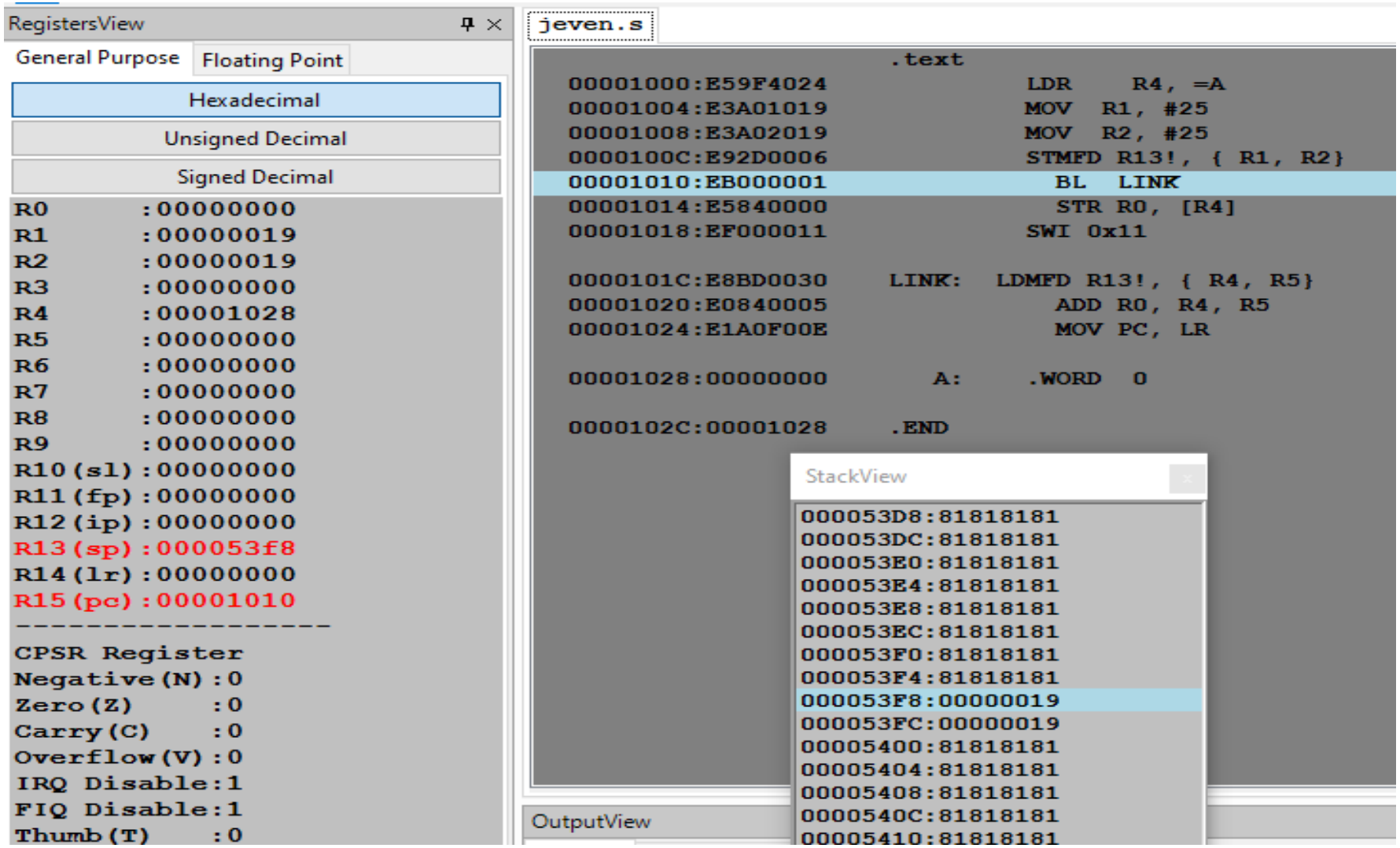
PARAMETER PASSING TO PROCEDURES USING STACK

```
        MOV   R1, #25                          ; parameter 1
        MOV   R2, #25                          ; parameter 2
        STMFD R13!, {R1, R2}                   ; parameters are PUSHed on stack.
    BL  LINK
        LDR   R4, =A
    STR R0, [R4]                               ; return value in Reg. R0.
        SWI 0x11

LINK:   LDMFD R13!, {R4, R5}                   ; parameters are POPed from the stack

        ADD R0, R4, R5                         ; Result is in register R0.
        MOV PC, LR

  A:    .WORD 0

.END
```

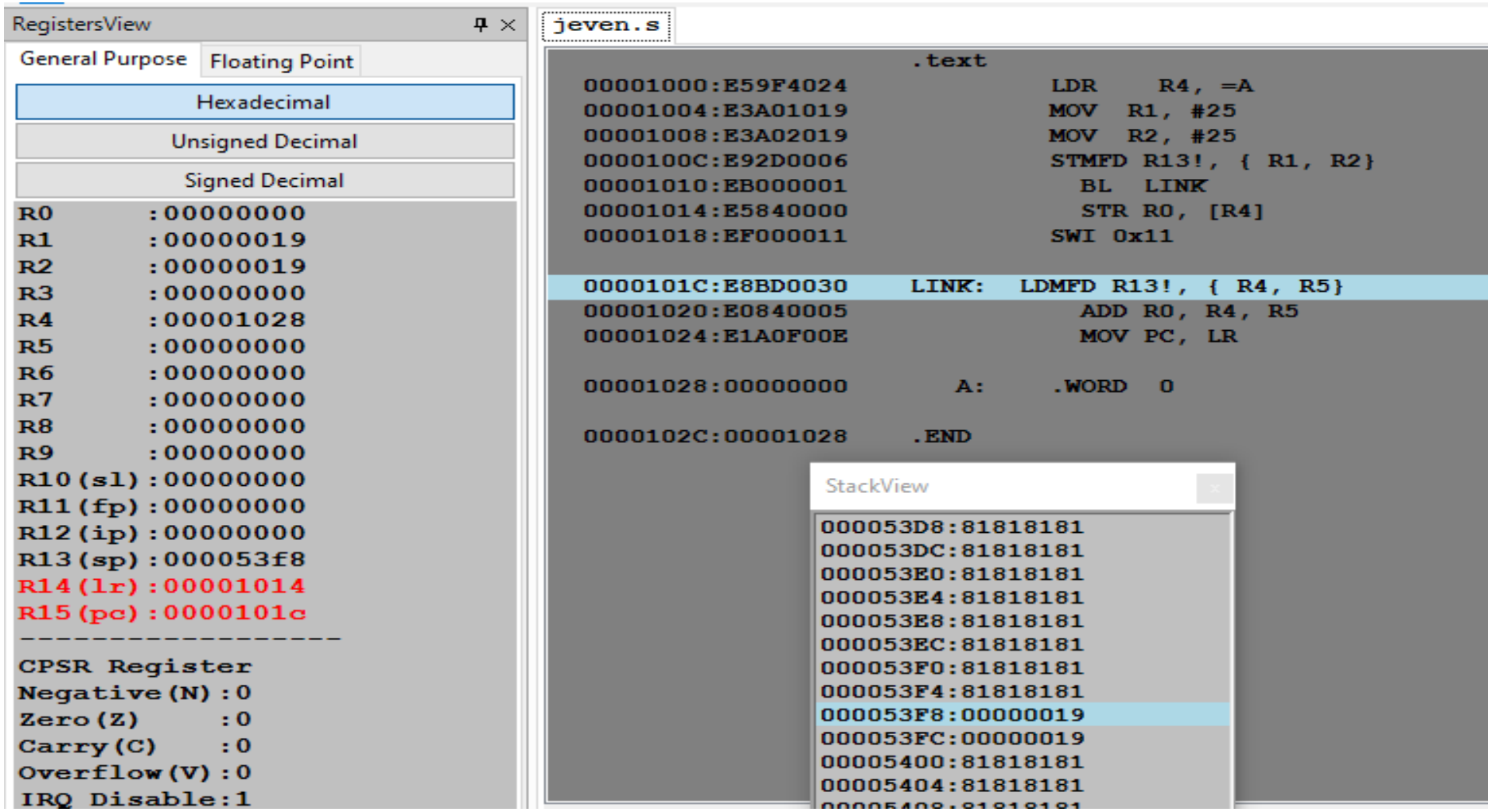
PARAMETER PASSI NG TO PROCEDURES USI NG STACK



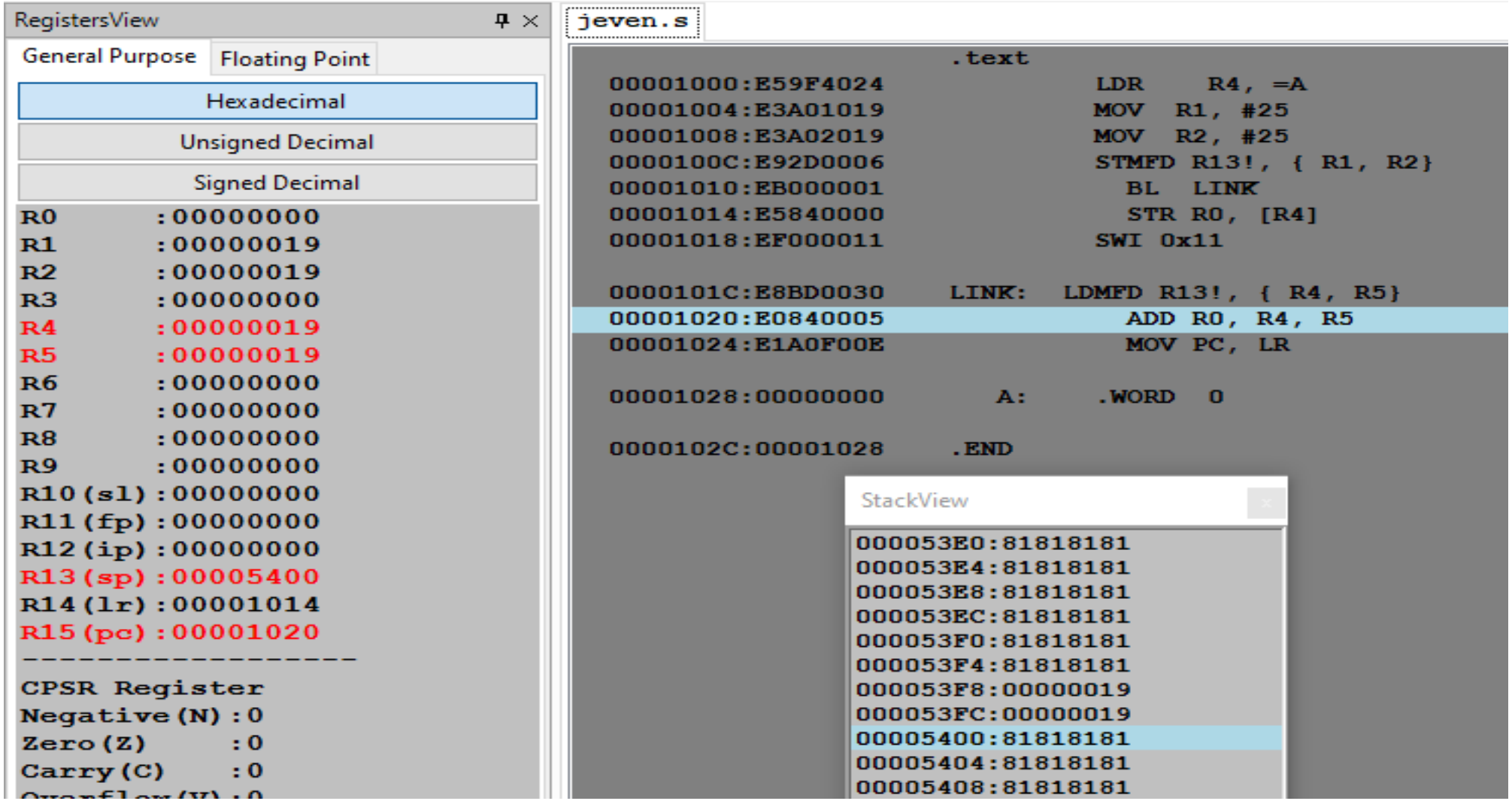
PARAMETER PASSING TO PROCEDURES USING STACK



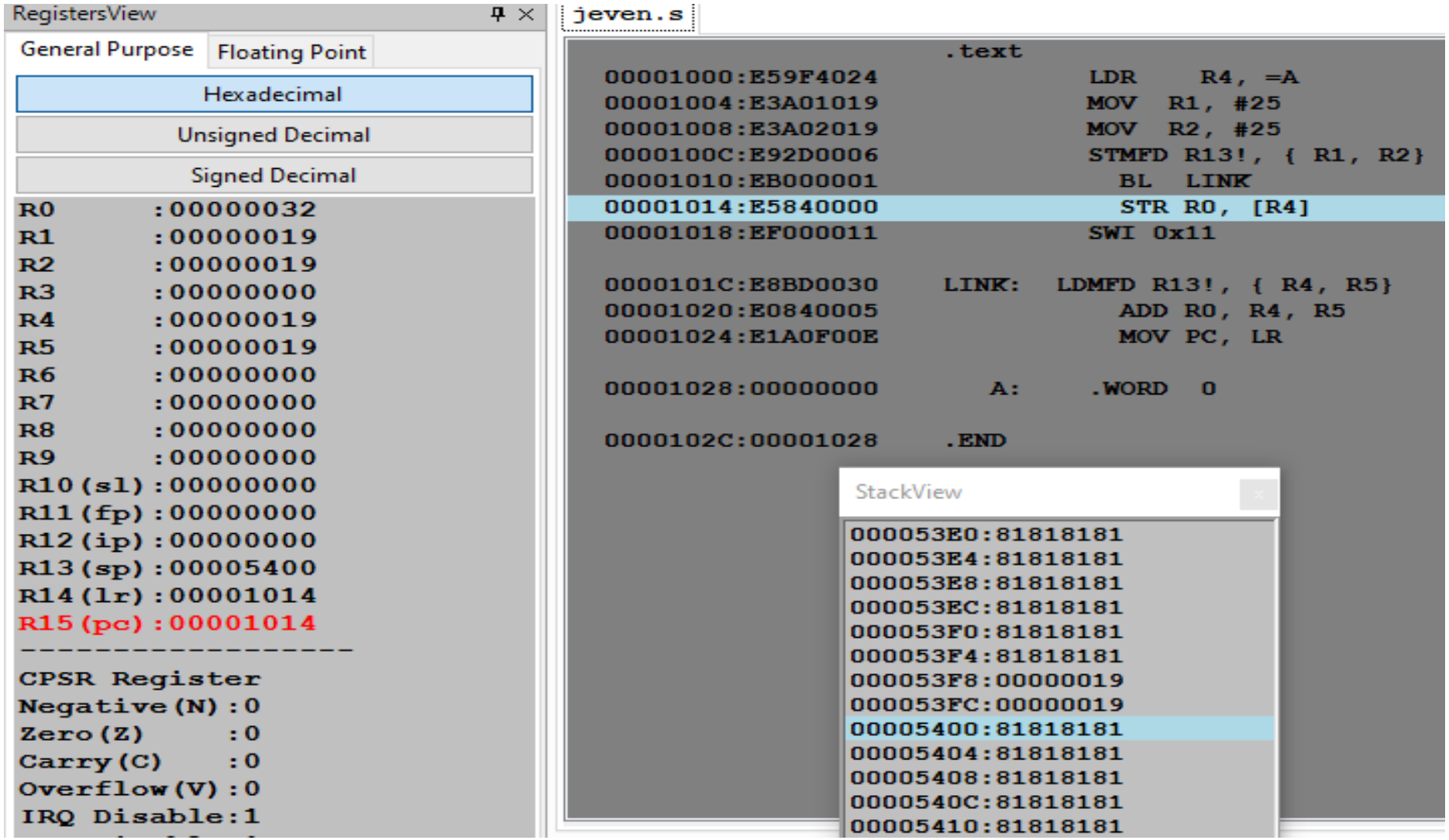
PARAMETER PASSING TO PROCEDURES USING STACK



PARAMETER PASSI NG TO PROCEDURES USI NG STACK



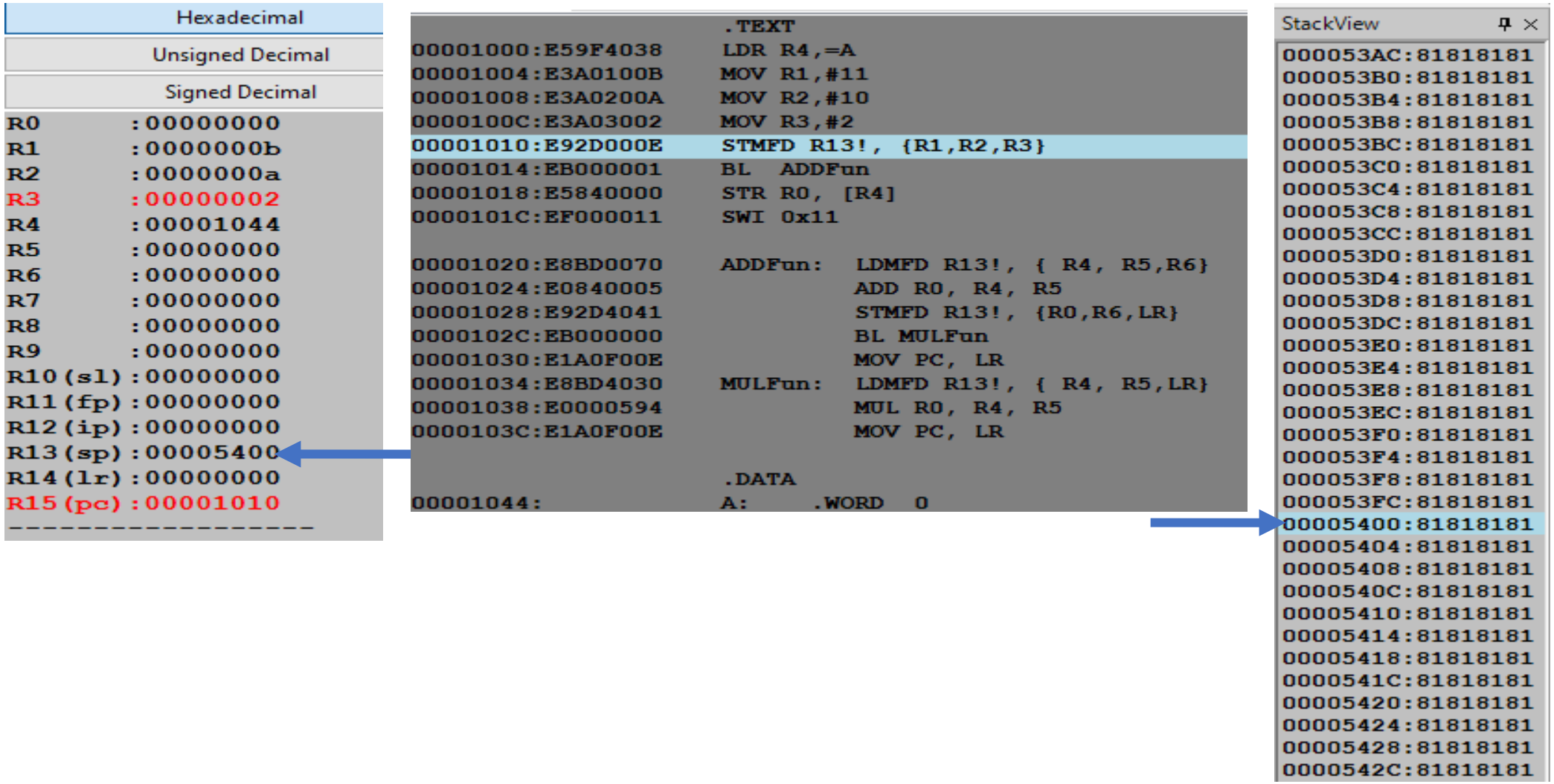
PARAMETER PASSING TO PROCEDURES USING STACK



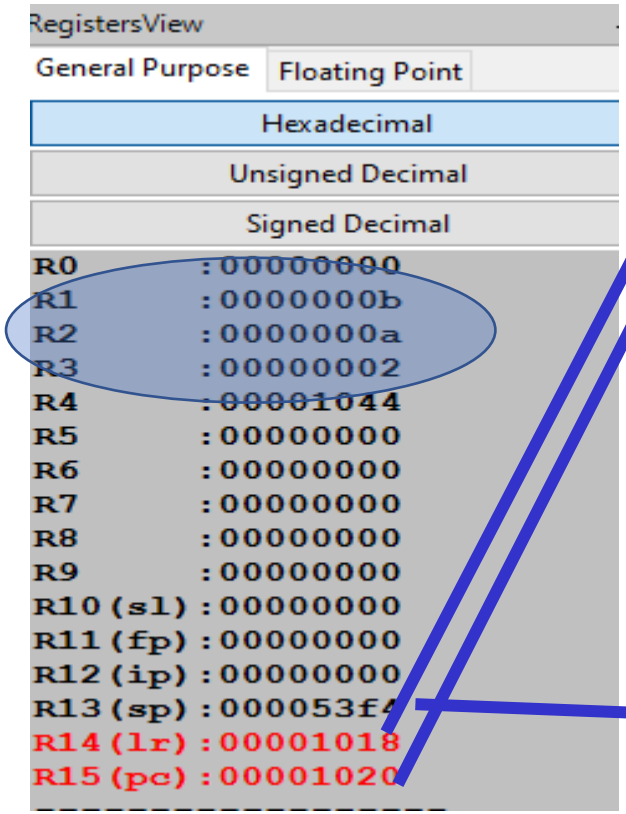
NESTED PROCEDURE CALL

```
.DATA
A:      .WORD   0

.TEXT           ; MAIN
Procedure
LDR R4, =A
MOV R1, #11
MOV R2, #10
MOV R3, #02
STMFD R13!, {R1, R2, R3}

BL  ADDFun          ; Call to
ADD Procedure
STR R0, [R4]

ADDFun:  LDMFD R13!, {R4, R5, R6}       ; ADD
Procedure
                ADD R0, R4, R5
                STMFD R13!, {R0, R6, LR}

                BL MULFun
            ; Call to MUL Procedure
MULFun  LDMFD R13!, {LR}{R4, R5}
                MOV PC, LR
            ; Return to Main Procedure
                MOV PC, LR
        ; Return to ADD Procedure
```

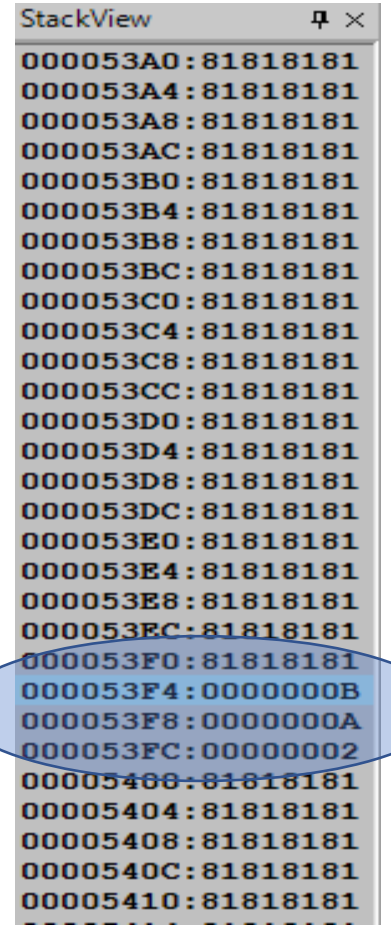
NESTED PROCEDURE CALL



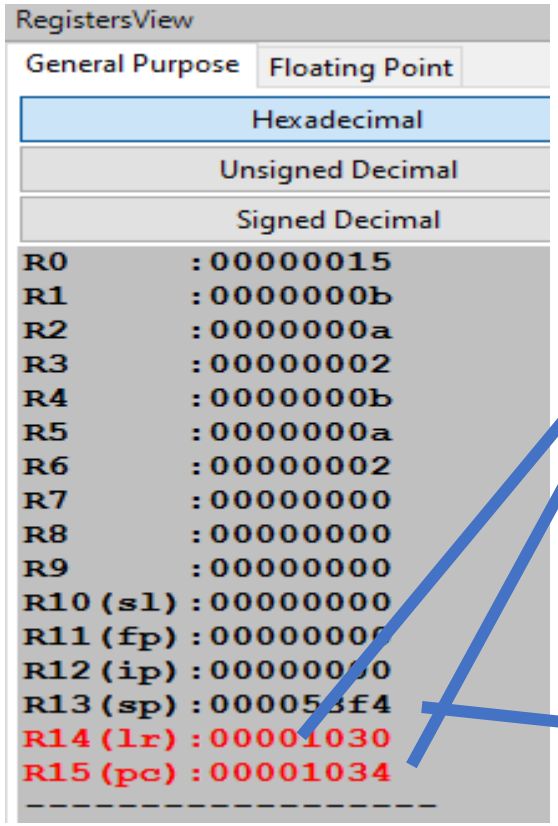
NESTED PROCEDURE CALL



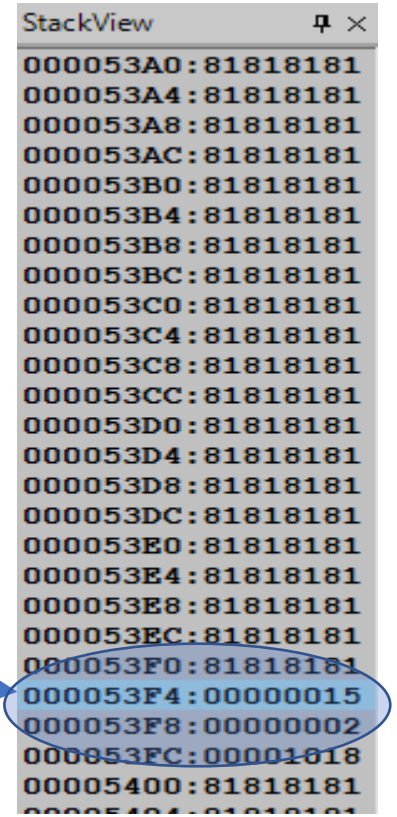

```
                        .TEXT
00001000:E59F4038       LDR R4,=A
00001004:E3A0100B       MOV R1,#11
00001008:E3A0200A       MOV R2,#10
0000100C:E3A03002       MOV R3,#2
00001010:E92D000E       STMFD R13!, {R1,R2,R3}
00001014:EB000001       BL  ADDProc
00001018:E5840000       STR R0, [R4]
0000101C:EF000011       SWI 0x11
00001020:E8BD0070       ADDProc:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                 ADD R0, R4, R5
00001028:E92D4041                 STMFD R13!, {R0,R6,LR}
0000102C:EB000000                 BL MULProc
00001030:E1A0F00E                 MOV PC, LR
00001034:E8BD4030       MULProc:  LDMFD R13!, { R4, R5,LR}
00001038:E0000594                 MUL R0, R4, R5
0000103C:E1A0F00E                 MOV PC, LR

                        .DATA
00001044:               A:      .WORD  0
```



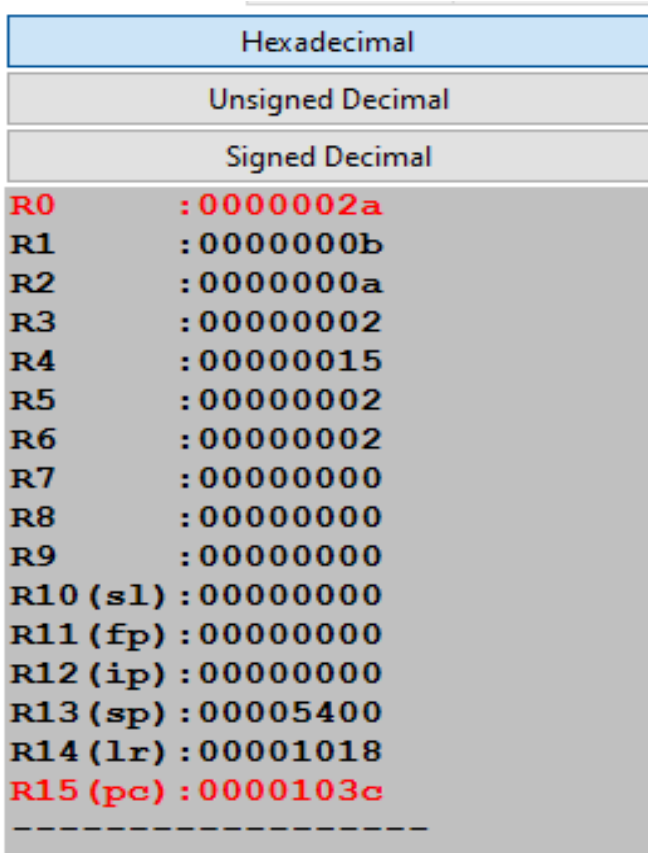
NESTED PROCEDURE CALL




```
                        .TEXT
00001000:E59F4038       LDR R4,=A
00001004:E3A0100B       MOV R1,#11
00001008:E3A0200A       MOV R2,#10
0000100C:E3A03002       MOV R3,#2
00001010:E92D000E       STMFD R13!, {R1,R2,R3}
00001014:EB000001       BL  ADDProc
00001018:E5840000       STR R0, [R4]
0000101C:EF000011       SWI 0x11
00001020:E8BD0070       ADDProc:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                 ADD R0, R4, R5
00001028:E92D4041                 STMFD R13!, {R0,R6,LR}
0000102C:EB000000                 BL MULProc
00001030:E1A0F00E                 MOV PC, LR
00001034:E8BD4030       MULProc:  LDMFD R13!, { R4, R5,LR}
00001038:E0000594                 MUL R0, R4, R5
0000103C:E1A0F00E                 MOV PC, LR

                        .DATA
00001044:               A:      .WORD  0
```

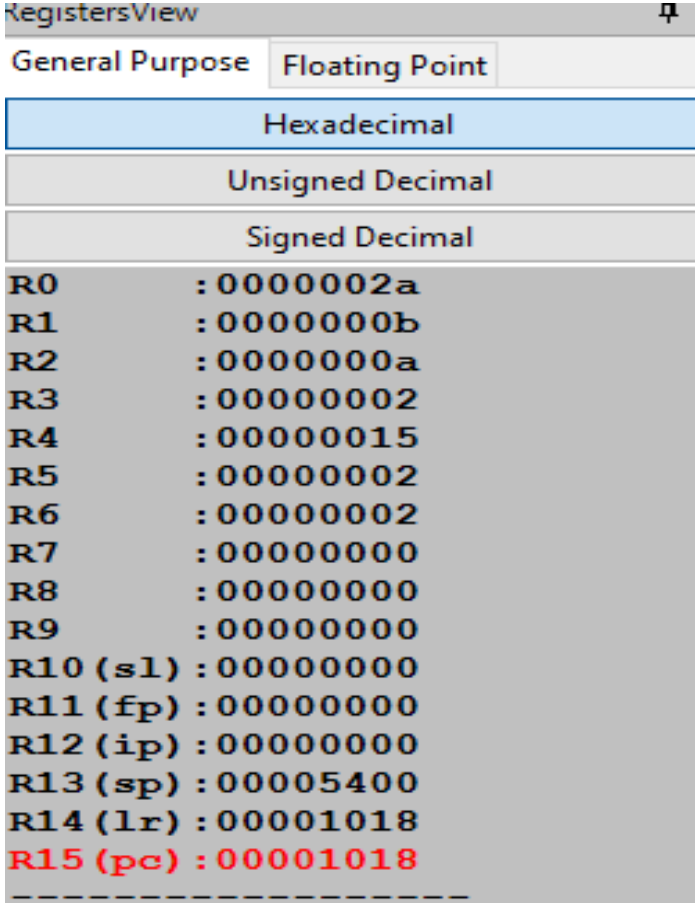


NESTED PROCEDURE CALL




```
                        .TEXT
00001000:E59F4038       LDR R4,=A
00001004:E3A0100B       MOV R1,#11
00001008:E3A0200A       MOV R2,#10
0000100C:E3A03002       MOV R3,#2
00001010:E92D000E       STMFD R13!, {R1,R2,R3}
00001014:EB000001       BL  ADDProc
00001018:E5840000       STR R0, [R4]
0000101C:EF000011       SWI 0x11
00001020:E8BD0070       ADDProc:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                 ADD R0, R4, R5
00001028:E92D4041                 STMFD R13!, {R0,R6,LR}
0000102C:EB000000                 BL MULProc
00001030:E1A0F00E                 MOV PC, LR
00001034:E8BD4030       MULProc:  LDMFD R13!, { R4, R5,LR}
00001038:E0000594                 MUL R0, R4, R5
0000103C:E1A0F00E                 MOV PC, LR

                        .DATA
00001044:               A:     .WORD  0
```

NESTED PROCEDURE CALL




```
                        .TEXT
00001000:E59F4038       LDR R4,=A
00001004:E3A0100B       MOV R1,#11
00001008:E3A0200A       MOV R2,#10
0000100C:E3A03002       MOV R3,#2
00001010:E92D000E       STMFD R13!, {R1,R2,R3}
00001014:EB000001       BL  ADDProc
00001018:E5840000       STR R0, [R4]
0000101C:EF000011       SWI 0x11
00001020:E8BD0070       ADDProc:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                 ADD R0, R4, R5
00001028:E92D4041                 STMFD R13!, {R0,R6,LR}
0000102C:EB000000                 BL MULProc
00001030:E1A0F00E                 MOV PC, LR
00001034:E8BD4030       MULProc:  LDMFD R13!, { R4, R5,LR}
00001038:E0000594                 MUL R0, R4, R5
0000103C:E1A0F00E                 MOV PC, LR

                        .DATA
00001044:               A:     .WORD  0
```

Microprocessor & Computer Architecture (µpCA)

UE24CS251B

Software Interrupts

Session - 8.2

WHAT IS INTERRUPT/ EXCEPTION?

- Main ()
- {
- :
- Doing something
- (e.g.
- browsing)
- :
- } ring

Can happen anytime
Depends on types of interrupts

Phone rings

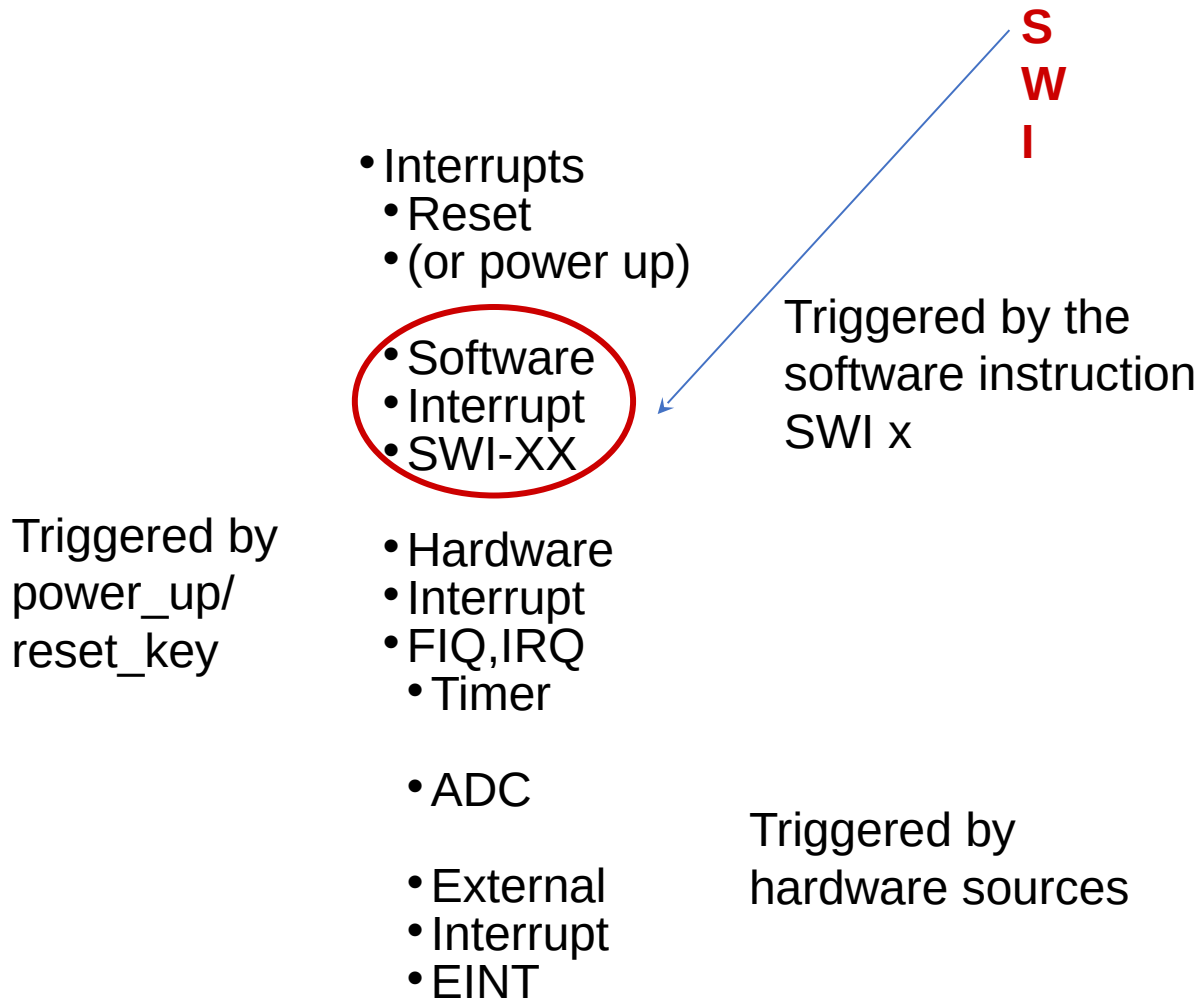
Phone rings

_isr() // Interrupt service routine
{

some tasks (e.g. answer telephone)

} //when finished,
//goes back to main

IMPORTANT INTERRUPTS



INTERRUPTS AND EXCEPTION

- The terms are used differently by various manufacturers
- Traditionally exception means

The normal operation of a program is interrupted and the processor will execute another piece of software (exception handling) somewhere.

- Interrupt (hardware interrupt) is an exception caused by some hardware condition happening outside the processor (e.g. external hard interrupt, IRQ FIQ).
- Software interrupt (SWI) is an exception caused by an assembly software instruction (SWI 0x?? exception call instruction) written in the software code.
- Trap is an exception caused by a failure condition of the processor (e.g. abort "pre-fetch , data" , undefined instruction, divided_by_zero, or stack overflow etc)

IMPORTANT INTERRUPTS IN WORDS

- Reset, a special interrupt to start the system– happens at power up , or reset button depressed)

- Software interrupt SWI :

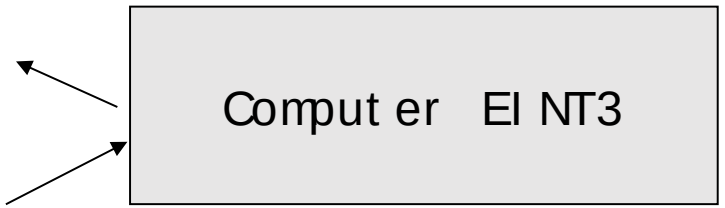
  Similar to subroutine – happens when "SWI 0x??" is written in the program

- Hardware interrupt
  - FIQ (fast interrupt) or IRQ (external interrupt), when
    - the external interrupt request pin is pulled low, or
    - an analog to digital conversion is completed, or
    - A timer/counter has made a regular request

HARDWARE AND SOFTWARE INTERRUPT

- **Hardware interrupt**

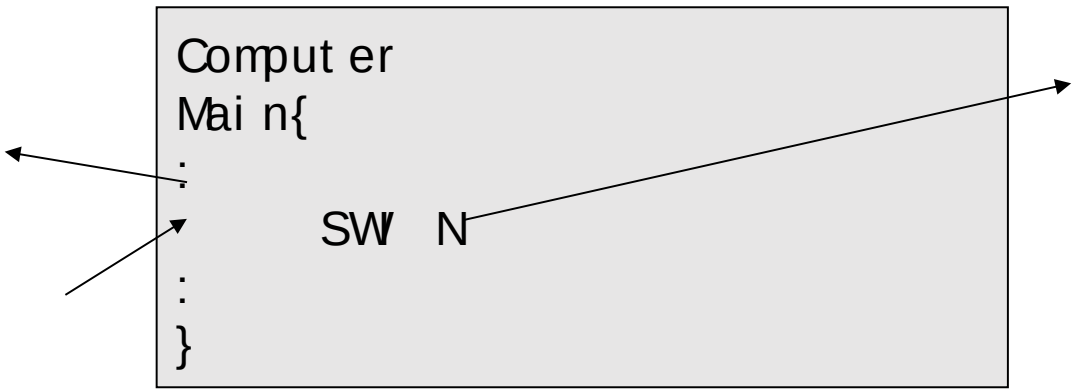
```
IRQ_Eint1(
)
{
::
}
```



A falling edge at EINT3 will trigger the execution of the interrupt service routine
__irq IRQ_Eint1()

- **Software interrupt**

```
N-th-sys-
routine()
{
::
}
```



An instruction “SWI N” in the program will trigger the
execution of
the N-th-sys-routine
(system routine)

EXCEPTION (INTERRUPT) MODES

- ARM supports 7 types of exceptions and has a privileged processor mode for each type of exception.
- ARM Exception (interrupt) vectors

INTERRUPT VECTOR TABLE:

SWI

	Address	Exception	Mode in Entry
1	0x00000000	Reset	Supervisor
2	0x00000004	Undefined instruction	Undefined
3	0x00000008	Software Interrupt	Supervisor
4	0x0000000C	Abort (prefetch)	Abort
5	0x00000010	Abort (data)	Abort
x	0x00000014	Reserved	Reserved
6	0x00000018	IRQ (external interrupt)	IRQ
7	0x0000001C	FIQ (fast interrupt)	FIQ

SWI INTERRUPT PROCEDURES (enter the supervisor mode)

- **SWI (software interrupt)**
  - Caused by **"SWI 0x??"** in your program
  - Arm completes the current instruction.
  - **Go to SWI** exception address **0x08** (short form for 0x 0000 0008)
  - {Change to supervisor op. mode : CPSR (bit 0-4)
  - Return from interrupt
    - **MOVS pc, lr**
  - return to main

SOFTWARE INTERRUPTS - 'SUPERVISOR CALL'

- Whenever a program requires **input or output**, for instance to send some **text to the display,** it is normal to **call a** supervisor routine.
  - **To read data from the keyboard or to display the contents on the console**
- **The supervisor is a program which operates at a privileged level, which means that it can do things that a user-level program cannot do directly.**
- The limitations on the capabilities of a user-level program vary from system to system, but in many systems the user cannot access hardware facilities directly.
- **The supervisor provides trusted ways to access system resources which appear to the user-level program rather like special subroutine accesses.**

SOFTWARE INTERRUPTS - 'SUPERVISOR CALL'

- The instruction set includes a special instruction, **SWI**, to call these functions, (SWI stands for '**Software Interrupt**', but is usually pronounced '**Supervisor Call**').
- Note: In privileged modes there is another register for each mode called **Saved Processor Status Register (SPSR)**.

The SPSR is used to save the current **CPSR** before changing modes.

- SWI interrupt occurs when the SWI instruction has been fetched and decoded successfully, and none of the other higher priority exceptions/interrupts have been flagged.
- Upon entry to the handler the CPSR will be set to SVC mode.
- Note: If a **SWI calls another SWI** (which is a common occurrence), then to avoid corruption, the link register (LR & SPSR) must be stacked away before branching to the nested SWI.

EXAMPLE 01

A procedure to compute the statement in high level language using ARM ALP.

```
if (R0==R1) R2++;
```

```
        MOV R0, #10
        MOV R1, #10
        BL GREAT
        SWI 0x11            ; terminate the program / logical end.

GREAT:  CMP R0, R1
        ADDEQ R2, R2, #1
        MOV PC, LR
```

EXAMPLE 02

A program to display a string on the screen using ARM ALP.

```
                    System.out.print (" Hello World");
        LDR     R1, =A
LOOP:   LDRB    R0, [R1], #1
        CMP R0, #0
        SWINE 0x00                  ; display a character on the screen.
        BNE LOOP
        SWI   0x11                  ; terminate the program
.DATA
A:      .ASCIZ  "HELLO WORLD"
```

EXAMPLE 03

A procedure to display a string on the screen using ARM ALP.

```
                    // System.out.print (" Hello World");

                    LDR     R1, =A
    BL    strprint                    ; call display routine
                     SWI   0x11                          ; terminate the program.
strprint:   LDRB   R0, [R1], #1
                    CMP R0, #0
                    SWINE 0x00                  ; display a character on the
  screen.
                BNE    strprint
                    MOV PC, LR
.DATA
  A:     .ASCIZ    "HELLO WORLD"
```

EXAMPLE 04

A procedure to display a string on the screen using ARM ALP using routine 0x02.

```
        .TEXT
        LDR     R0, =A
        SWI     0x02            ; display a string on
the screen
        SWI     0x11            ; exit

        .DATA
A:      .ASCIZ  "HELLO WORLD"
```

MCQS

1. Which ARM instruction is used to call a subroutine, and what specific action does it perform regarding the return address?

A) `MOV PC, LR` – It jumps to the subroutine and saves the current Program Counter in R0.
B) `BL label` – It branches to the label and stores the address of the next instruction in the Link Register (R14).
C) `SWI 0x11` – It triggers a software interrupt to jump to a subroutine.
D) `BX LR` – It branches to the subroutine without saving the return address.

2. In a nested procedure call (where Subroutine A calls Subroutine B), what crucial step must be taken to prevent the loss of the return address?

A) The Stack Pointer (SP) must be reset to zero.
B) The Link Register (LR) must be pushed onto the stack before the second call is made.
C) The Program Counter (PC) must be manually incremented.
D) The CPSR register must be cleared.

MCQS

1. Which ARM instruction is used to call a subroutine, and what specific action does it perform regarding the return address?

A) `MOV PC, LR` – It jumps to the subroutine and saves the current Program Counter in R0.
B) `BL label` – It branches to the label and stores the address of the next instruction in the Link Register (R14).
C) `SWI 0x11` – It triggers a software interrupt to jump to a subroutine.
D) `BX LR` – It branches to the subroutine without saving the return address.

2. In a nested procedure call (where Subroutine A calls Subroutine B), what crucial step must be taken to prevent the loss of the return address?

A) The Stack Pointer (SP) must be reset to zero.
B) The Link Register (LR) must be pushed onto the stack before the second call is made.
C) The Program Counter (PC) must be manually incremented.
D) The CPSR register must be cleared.

3. Which register is designated as the Stack Pointer in ARM architecture, and which instruction is typically used to pop data from the stack?**
A) `R14` is the Stack Pointer; `LDR` is used to pop data.
B) `R13` is the Stack Pointer; `LDMFD` is used to pop data.
C) `R15` is the Stack Pointer; `STMFD` is used to pop data.
D) `R12` is the Stack Pointer; `MOV` is used to pop data.

4. What is the fundamental difference between a Hardware Interrupt and a Software Interrupt (SWI)?
A) Hardware interrupts are triggered by external hardware events (like a timer), while SWIs are triggered by executing a specific instruction in the code.
B) Hardware interrupts always stop the processor, while SWIs run in the background.
C) Hardware interrupts use the stack, while SWIs do not.
D) SWIs are caused by errors (like division by zero), while Hardware interrupts are caused by user input.

3. Which register is designated as the Stack Pointer in ARM architecture, and which instruction is typically used to pop data from the stack?
A) `R14` is the Stack Pointer; `LDR` is used to pop data.
B) `R13` is the Stack Pointer; `LDMFD` is used to pop data.
C) `R15` is the Stack Pointer; `STMFD` is used to pop data.
D) `R12` is the Stack Pointer; `MOV` is used to pop data.

4. What is the fundamental difference between a Hardware Interrupt and a Software Interrupt (SWI)?
A) Hardware interrupts are triggered by external hardware events (like a timer), while SWIs are triggered by executing a specific instruction in the code.
B) Hardware interrupts always stop the processor, while SWIs run in the background.
C) Hardware interrupts use the stack, while SWIs do not.
D) SWIs are caused by errors (like division by zero), while Hardware interrupts are caused by user input.

5. When a Software Interrupt (SWI) occurs, into which mode does the processor switch, and to which vector address does it jump?

A) It stays in User mode and jumps to `0x00000000`.
B) It switches to Supervisor (SVC) mode and jumps to `0x00000008`.
C) It switches to IRQ mode and jumps to `0x00000018`.
D) It switches to FIQ mode and jumps to `0x0000001C`.

5. When a Software Interrupt (SWI) occurs, into which mode does the processor switch, and to which vector address does it jump?

A) It stays in User mode and jumps to `0x00000000`.
B) It switches to Supervisor (SVC) mode and jumps to `0x00000008`.
C) It switches to IRQ mode and jumps to `0x00000018`.
D) It switches to FIQ mode and jumps to `0x0000001C`.

THANK YOU

PES UNIVERSITY

Team MPCA
Department of Computer Science and Engineering